

```

{
    [ ] ( = - + ÷
    + [ ] * / )
}
plt.figure()
sns.histplot(data,
    bins=30,
    kde=True,
    color='coral',
    hue=plot,
)

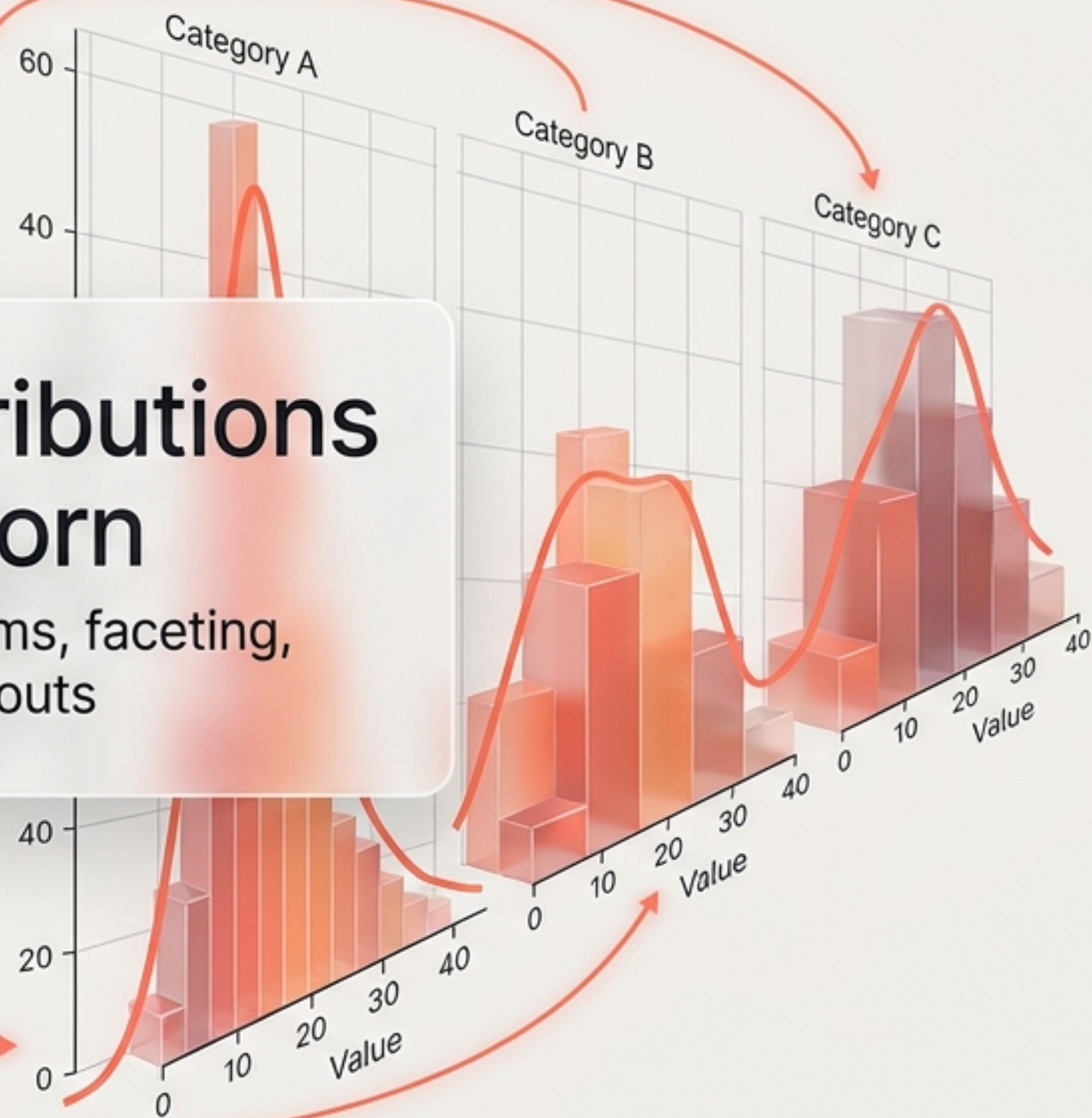
plt.figure()
g = sns.FacetGrid(
    col='category',
    height=5,
    aspect=1.5,
    hue=5
)
g.map(g, g)

plt.show()
{ = } [
+ - ) - * / + (
/ / .
}

```

Mastering Distributions with Seaborn

A visual guide to histograms, faceting, and multi-plot layouts

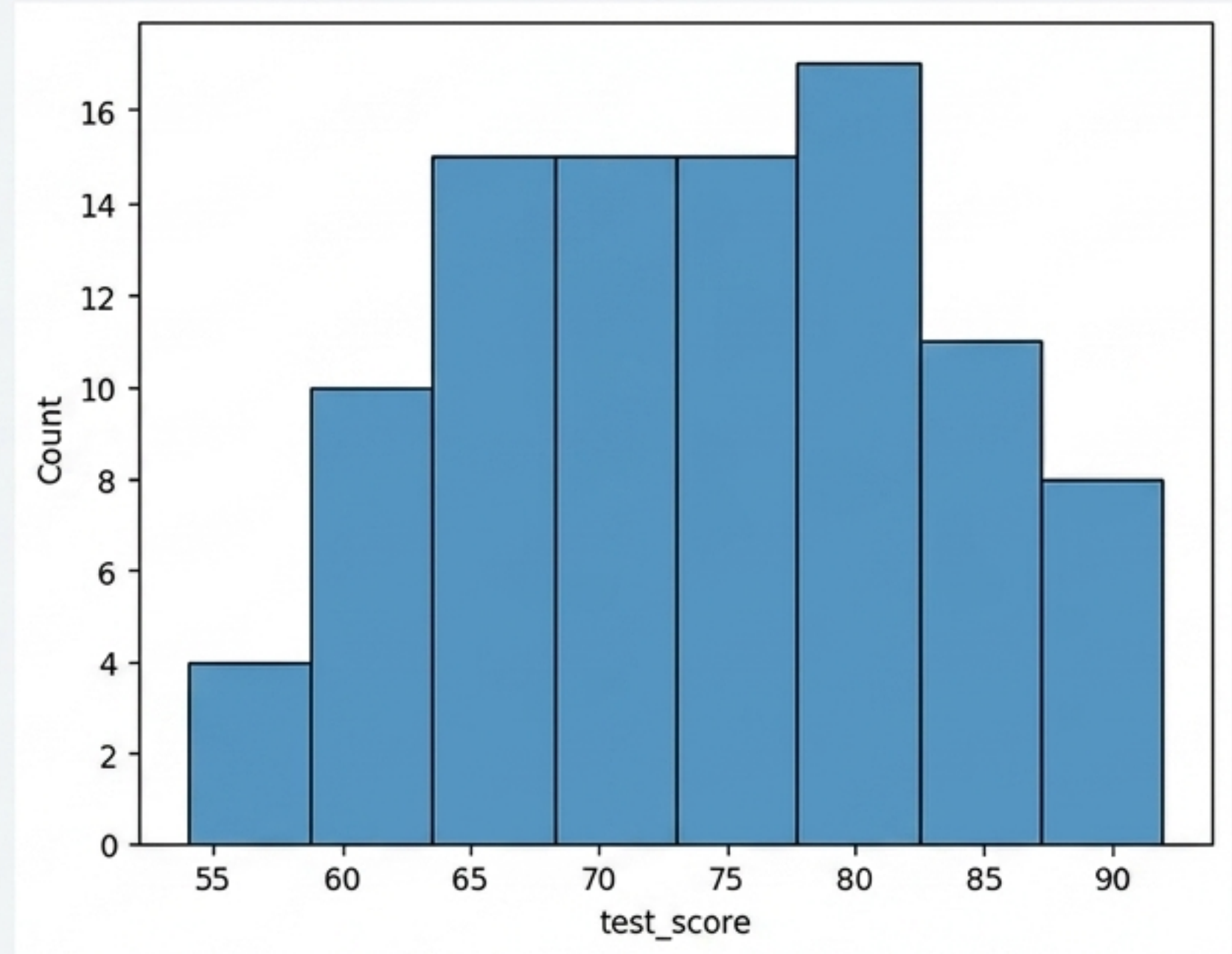


01 / The Foundation

```
sns.histplot(data=student, x="test_score")
```

Axes-Level Function

histplot automatically calculates bin sizes and counts the frequency of the target variable. It is the fundamental building block for continuous data distribution.

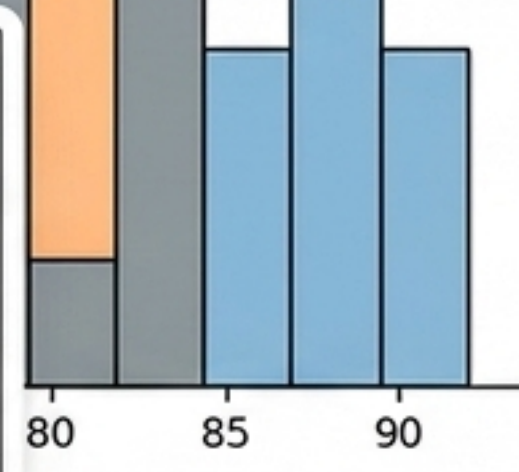
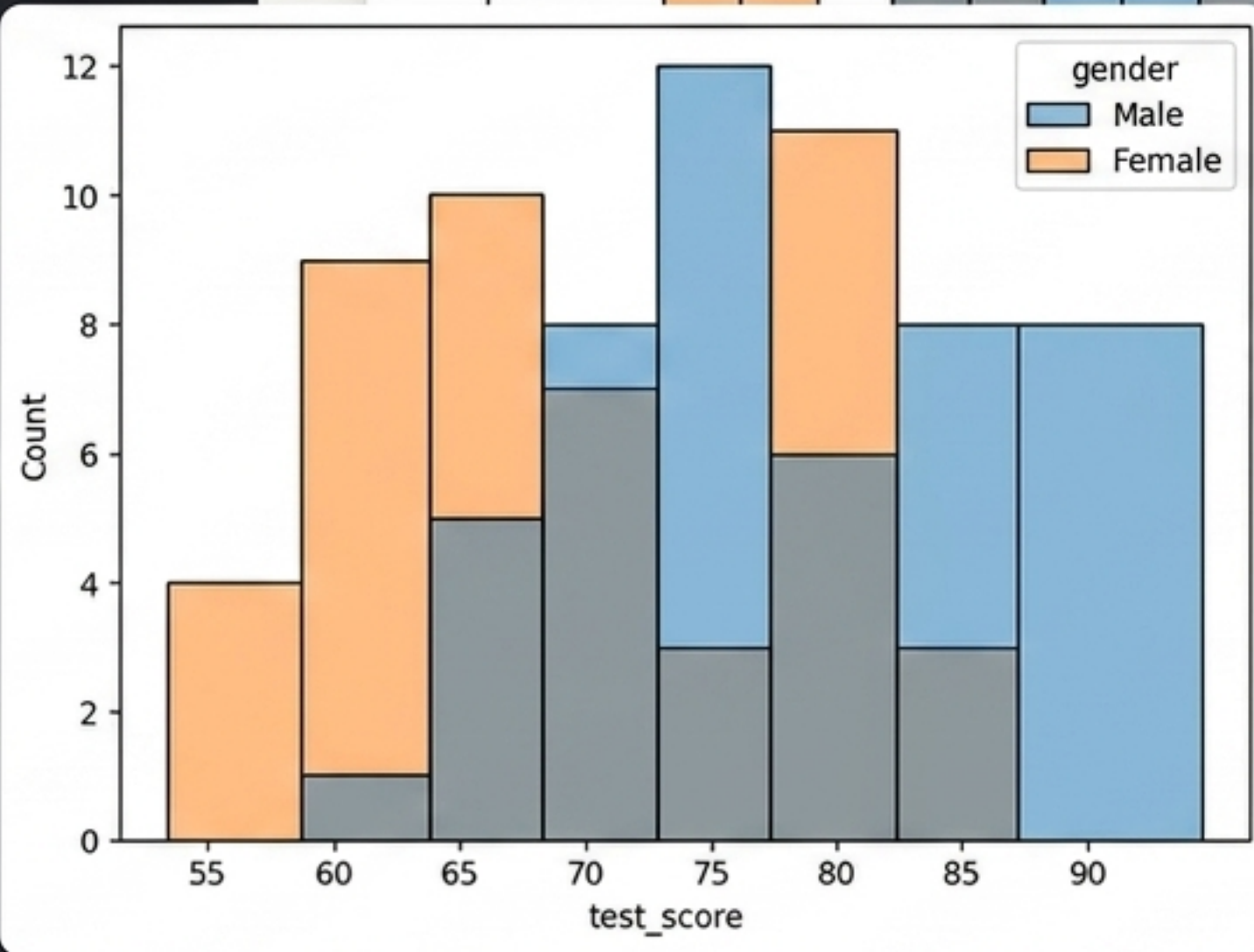
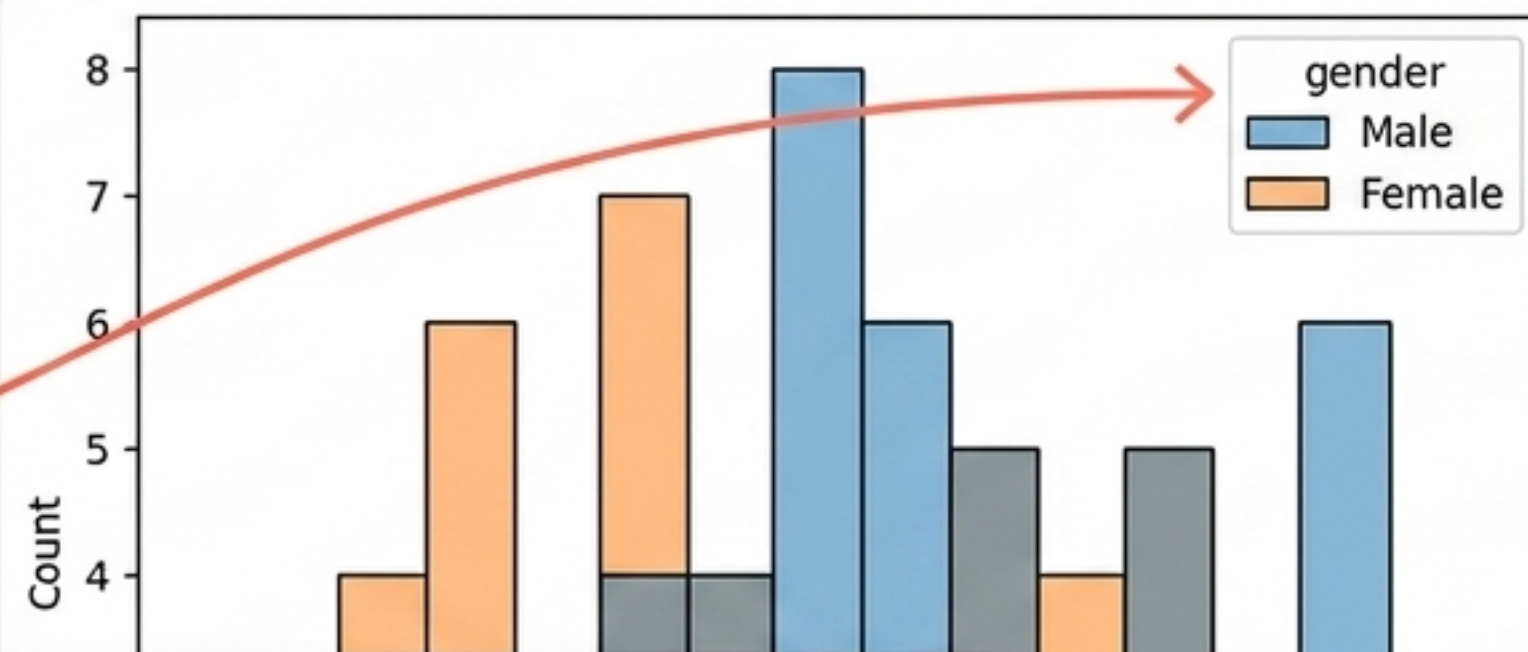


02 / Stratification

```
sns.histplot(data=student,  
             x="test_score",  
             hue="gender")
```

Grouping with hue

The hue parameter splits the dataset into distinct subgroups within the same axes. Overlapping areas are handled by default using transparency and blending.



03 / Granularity

A

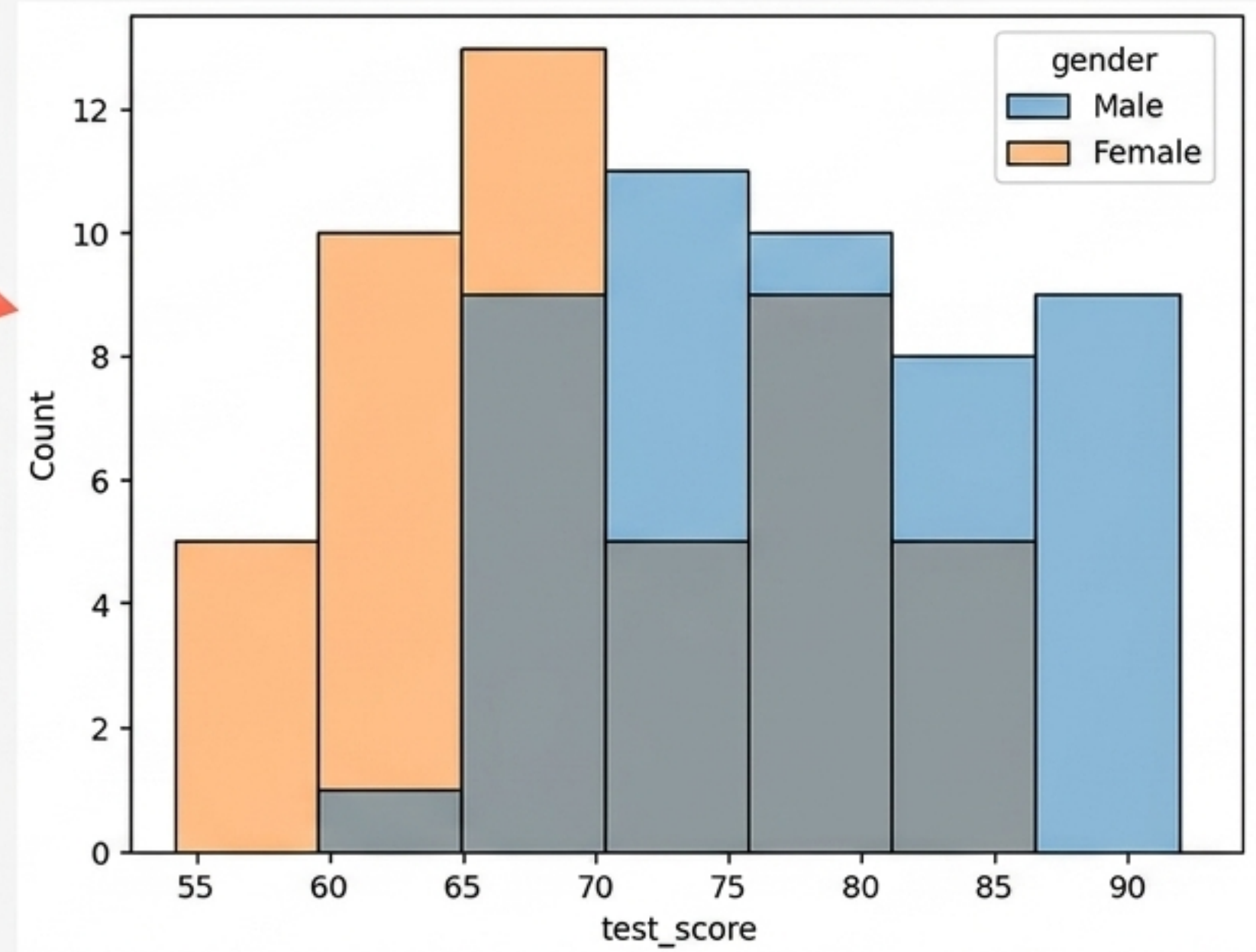
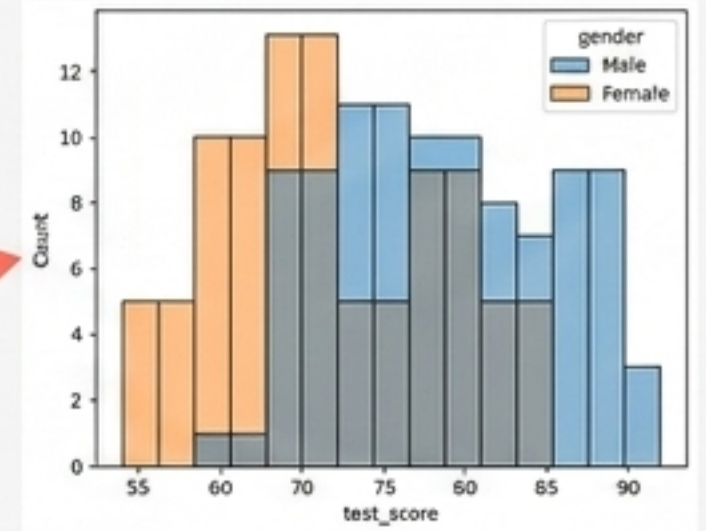
```
sns.histplot(..., hue="gender", bins=15)
```

B

```
sns.histplot(..., hue="gender", bins=7)
```

Controlling Bins

Lowering the bin count aggregates data into wider ranges. This smooths out noise but obscures fine detail. Higher bin counts reveal tighter, more specific clusters.



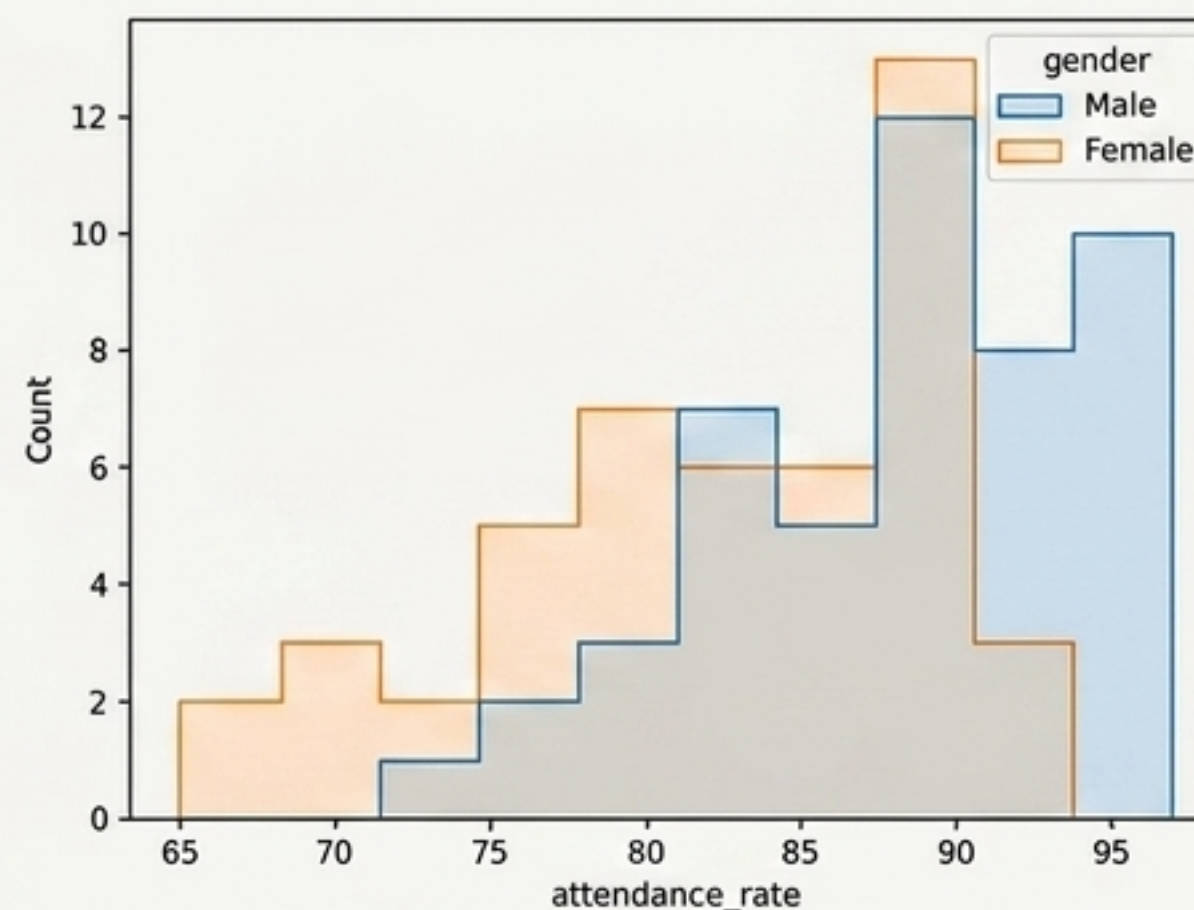
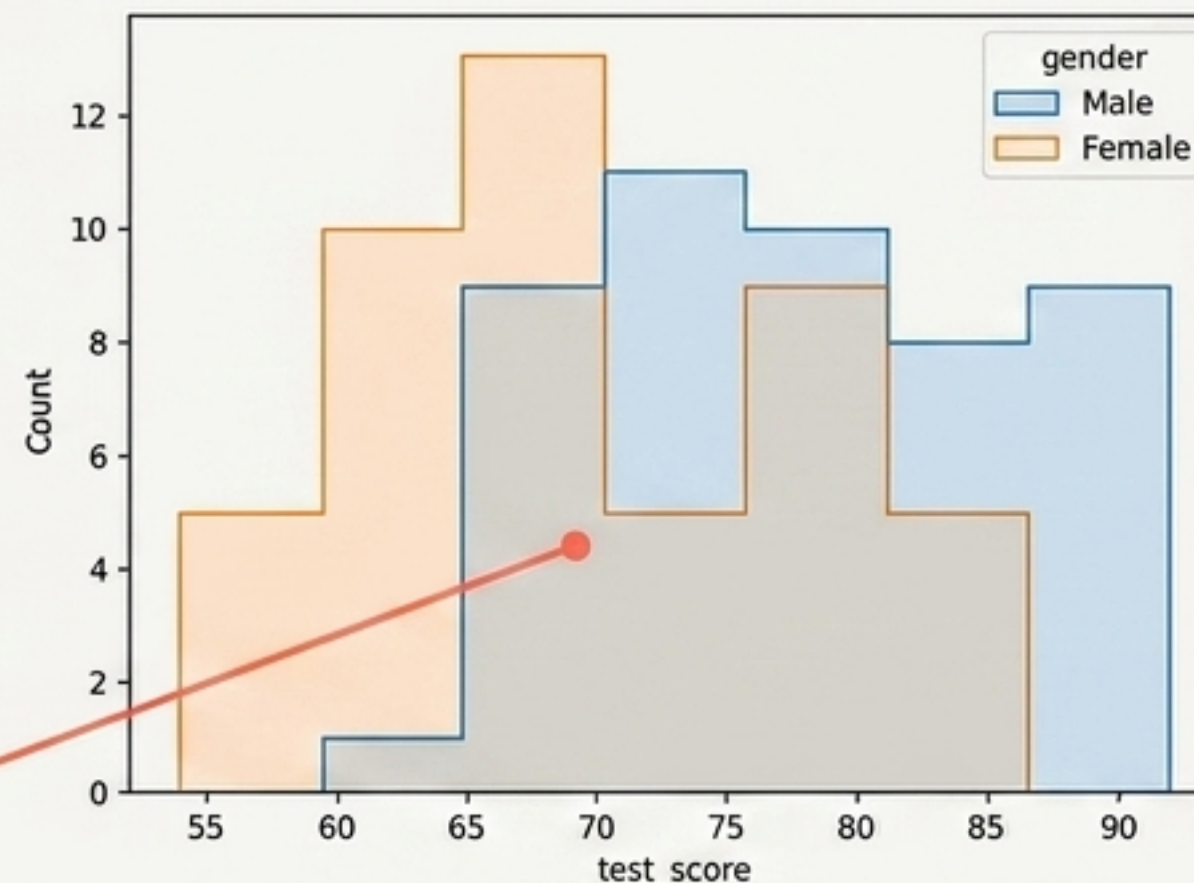
04 / Aesthetics

```
sns.histplot(...,  
             hue="gender",  
             bins=7,  
             element='step')
```

```
sns.histplot(...,  
             x="attendance_rate",  
             hue="gender",  
             bins=10,  
             element='step')
```

The Step Element

Changing the visual geometry from solid bars to outlines (step) removes internal dividers. This dramatically improves readability for overlapping distributions.



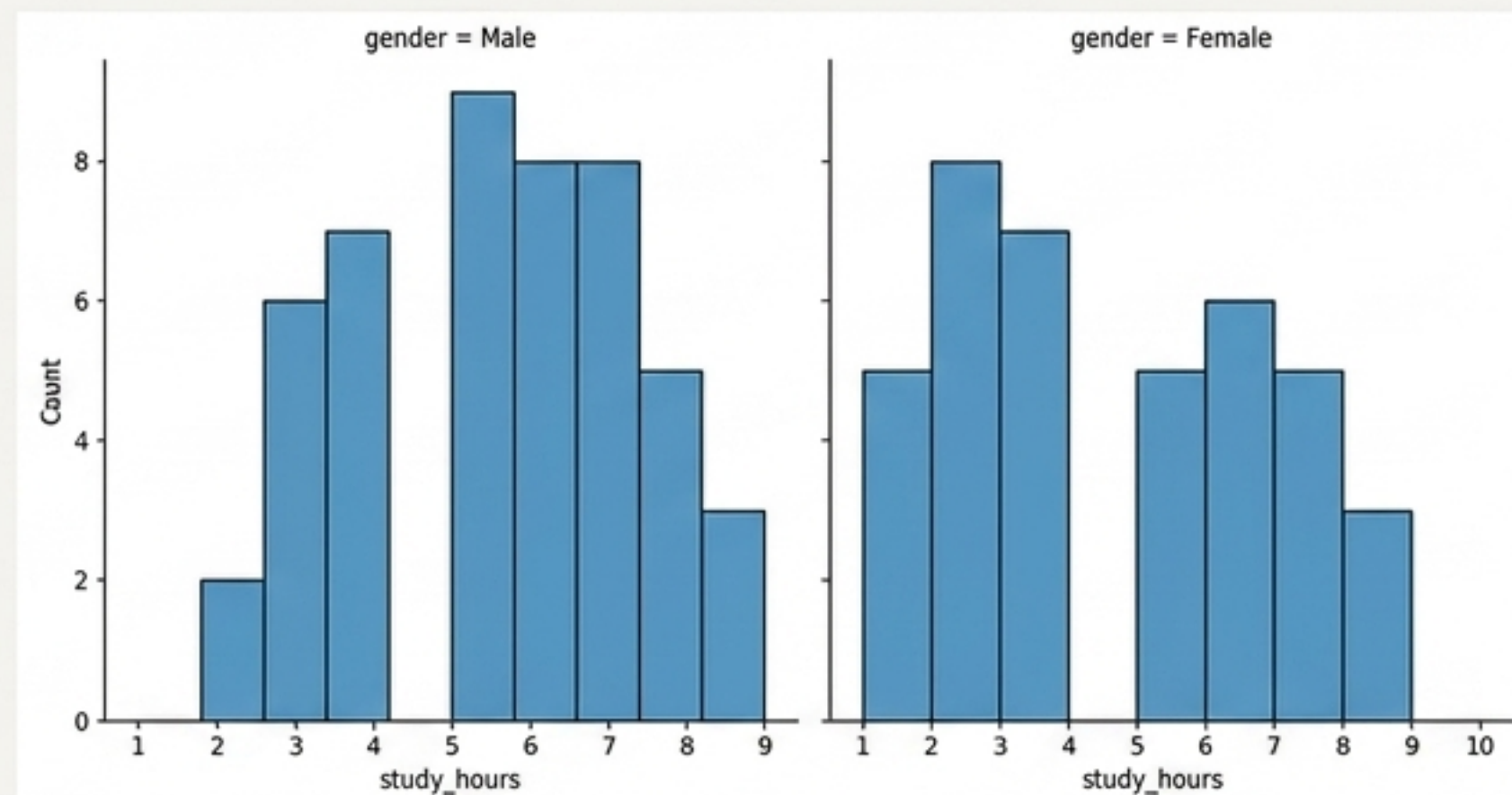
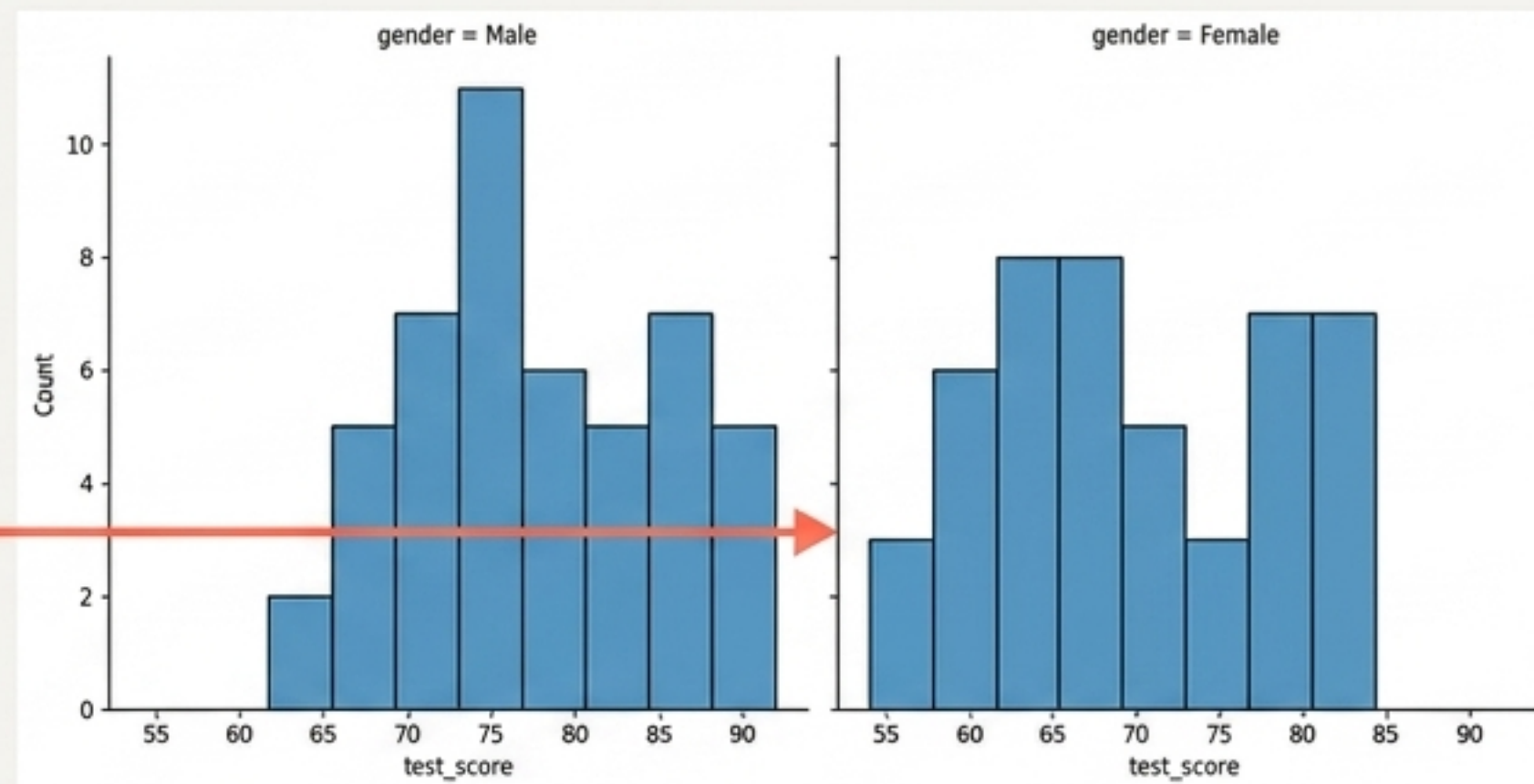
05 / Faceting

```
sns.displot(data=student, x="test_score",  
            bins=10, col="gender")
```

```
sns.displot(data=student, x="study_hours",  
            bins=10, col="gender")
```

Figure-Level Faceting

`displot` is a **Figure**-level function. Instead of overlapping data on one axis, `col="gender"` generates instant "small multiples"—separate, synchronized subplots sharing exact axis limits for accurate side-by-side comparison.

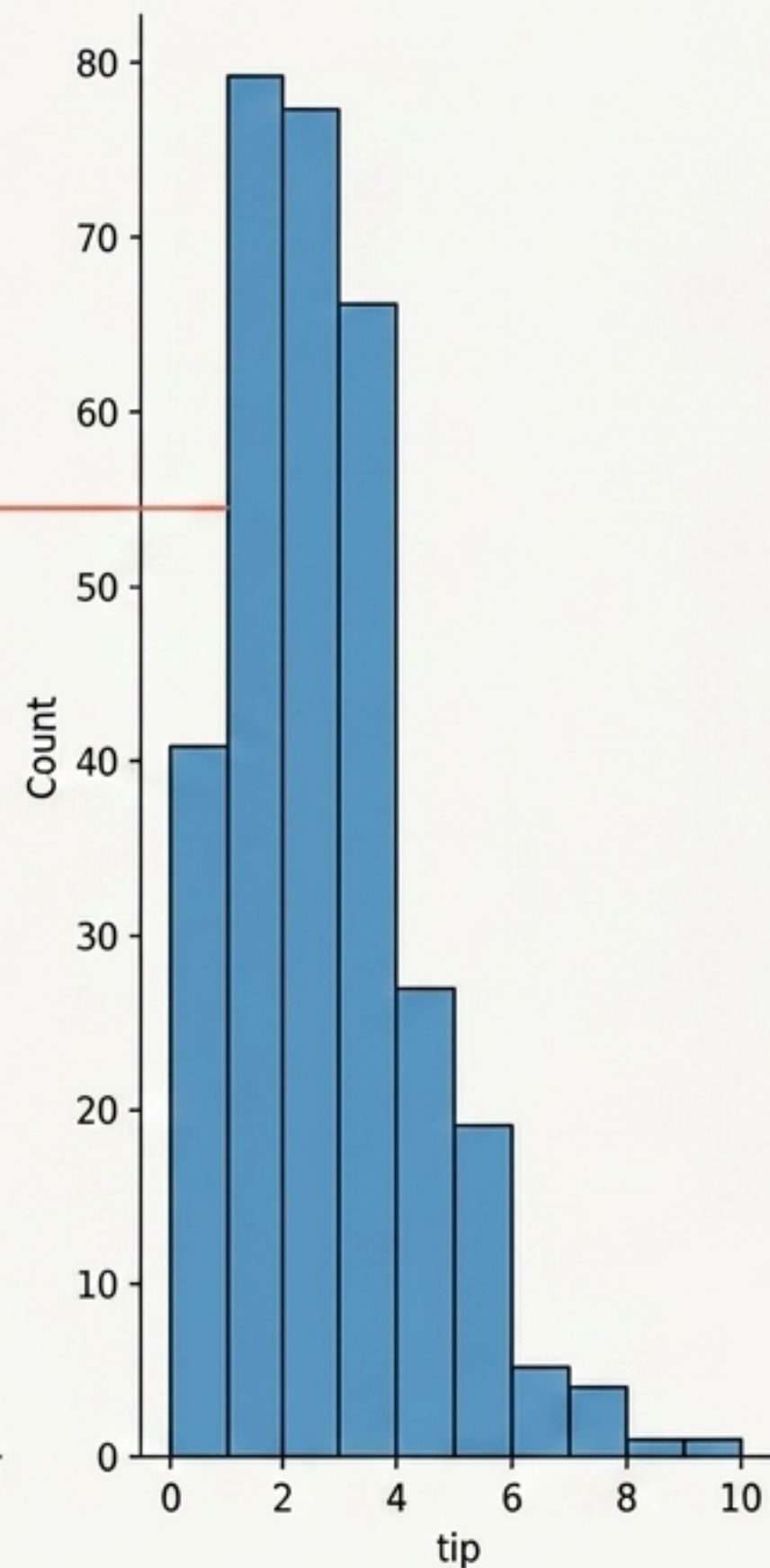
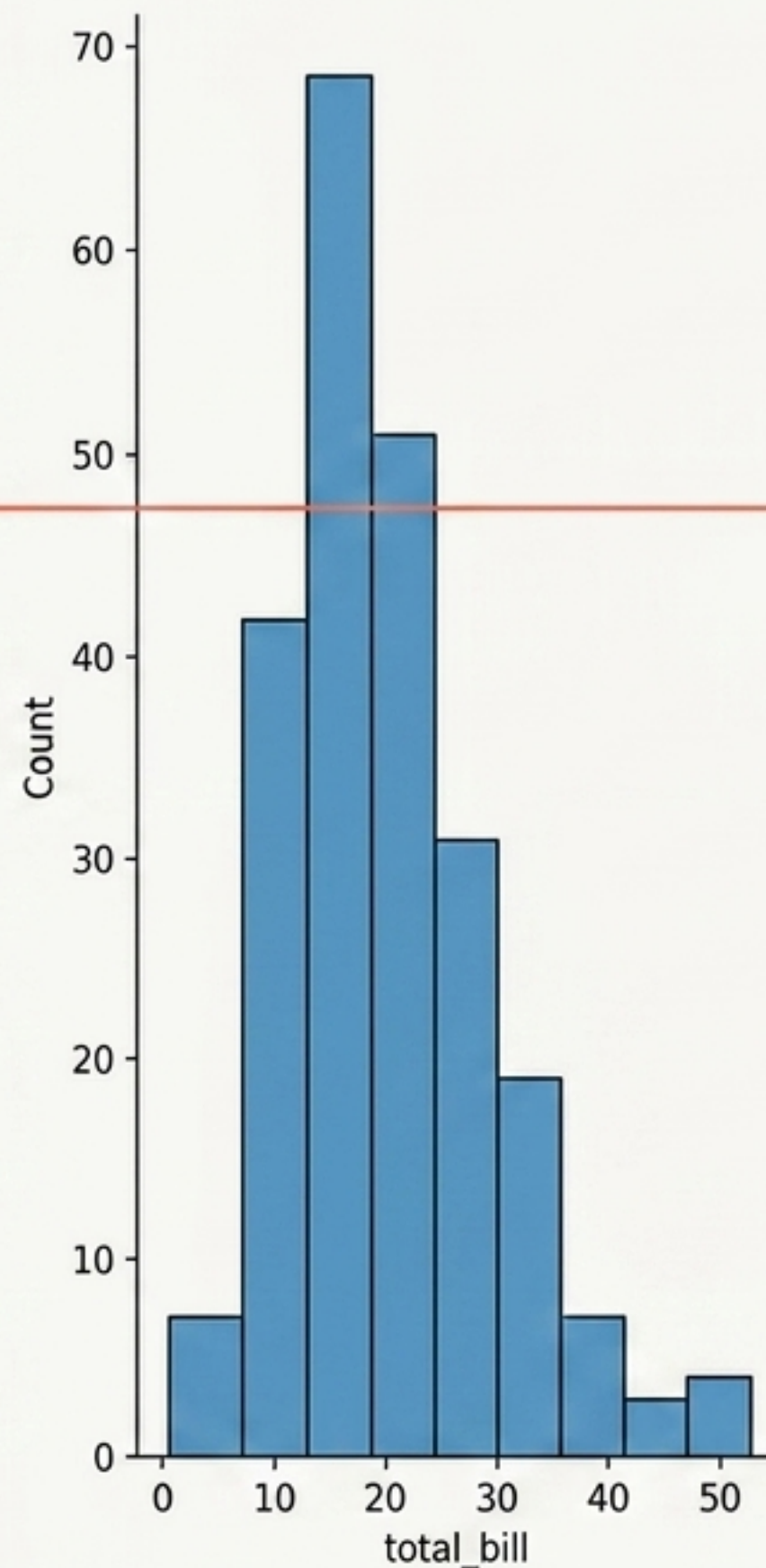


06 / Applying to New Contexts

```
sns.displot(data=tips_data, x='total_bill', bins=10)  
sns.displot(data=tips_data, x='tip', bins=10)
```

Validating the Technique

The exact same displot architecture instantly reveals the right-skewed behavior of restaurant spending and tipping in a completely different dataset structure.

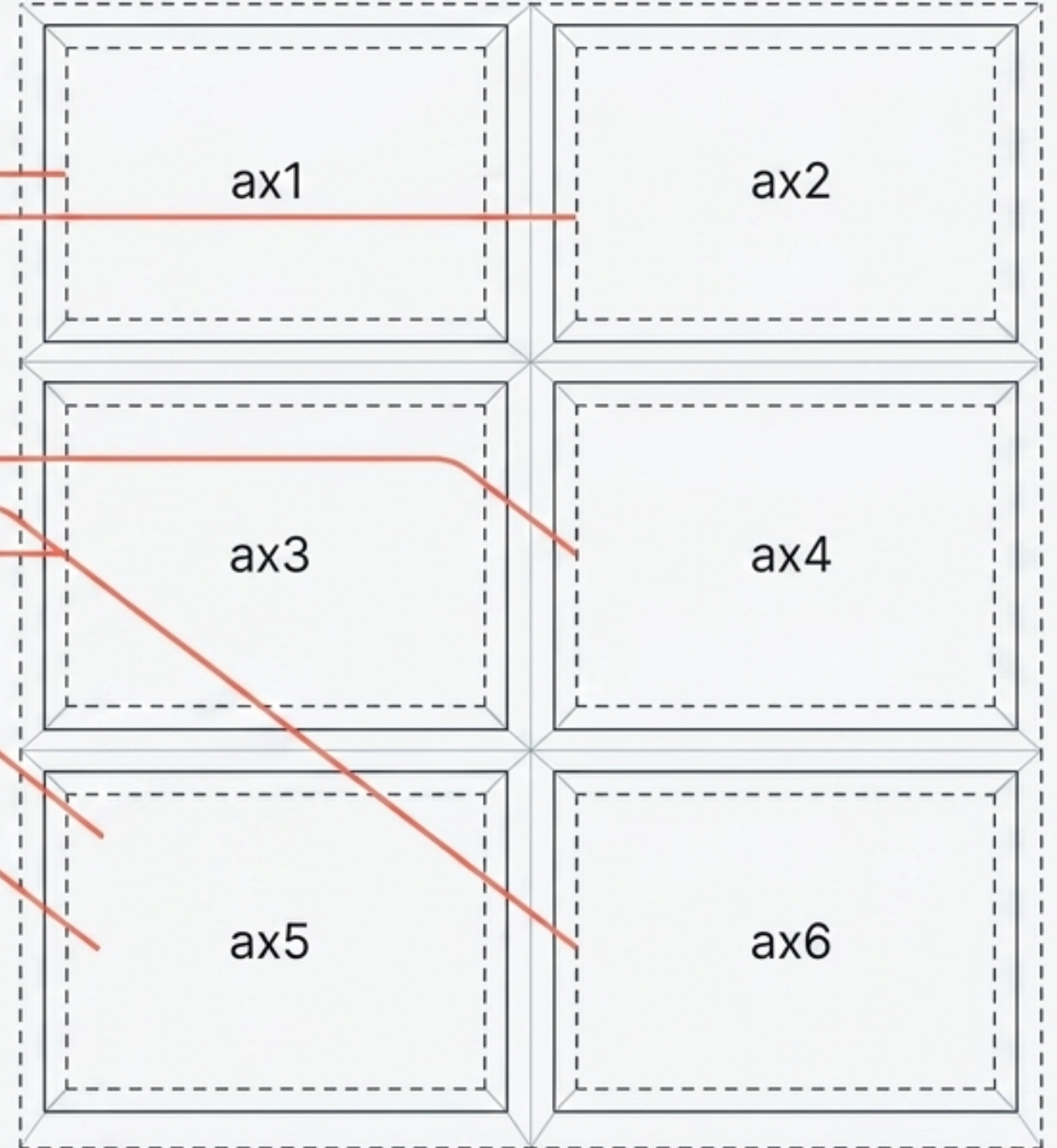


07 / Grid Setup

```
fig, ((ax1, ax2), (ax3, ax4), (ax5, ax6)) =  
    plt.subplots(3, 2, figsize=(12, 12))
```

The Multi-Plot Blueprint

Matplotlib's `subplots(3, 2)` dictates a grid of 3 rows and 2 columns. Tuple unpacking allows us to assign a specific variable name to each physical 'box' on the canvas.



08 / Compositing

```
sns.scatterplot(..., ax=ax1)
```

```
sns.lineplot(..., ax=ax2)
```

```
sns.lineplot(..., ax=ax3)
```

```
sns.histplot(..., ax=ax4)
```

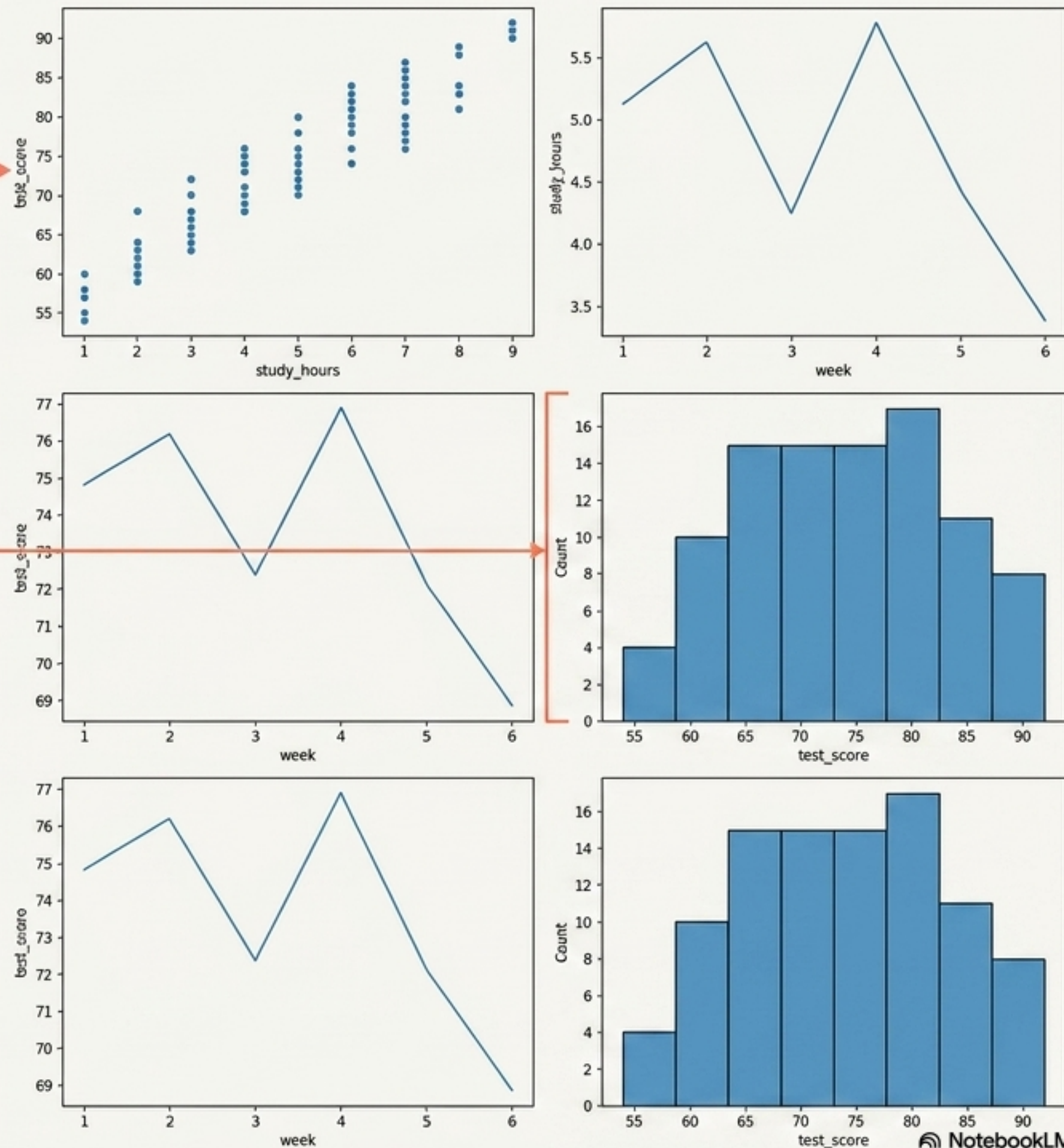
```
sns.histplot(..., ax=ax6)
```

```
sns.histplot(..., ax=ax6)
```

```
plt.tight_layout()
```

Populating the Dashboard

By using the `ax=` parameter, we can force individual axes-level plots into specific positions within our custom blueprint.



Synthesis: The Seaborn Matrix

Axes-Level Functions

Examples: `histplot()`, `scatterplot()`, `lineplot()`

Behavior: Draws onto a single, specific matplotlib axis.

Superpower: Precision compositing. Perfect for slotting into custom multi-plot dashboards via the `ax=` parameter.

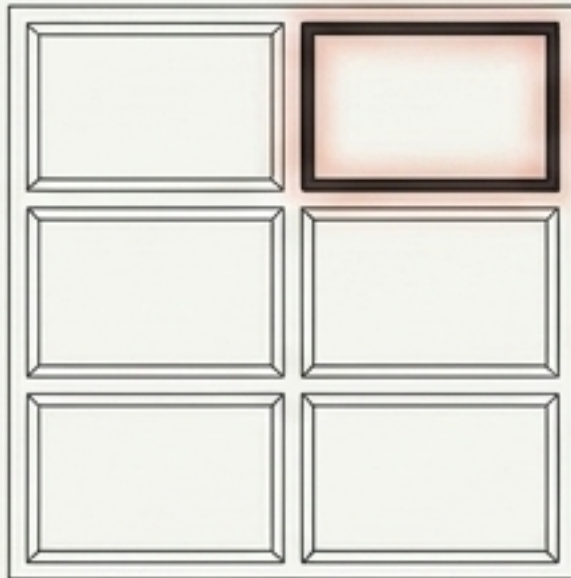


Figure-Level Functions

Examples: `displot()`, `relplot()`, `catplot()`

Behavior: Initializes its own complete Figure environment.

Superpower: Instant faceting. Exclusive access to `col=` or `row=` parameters to generate synchronized small multiples grids without manual wireframing.

